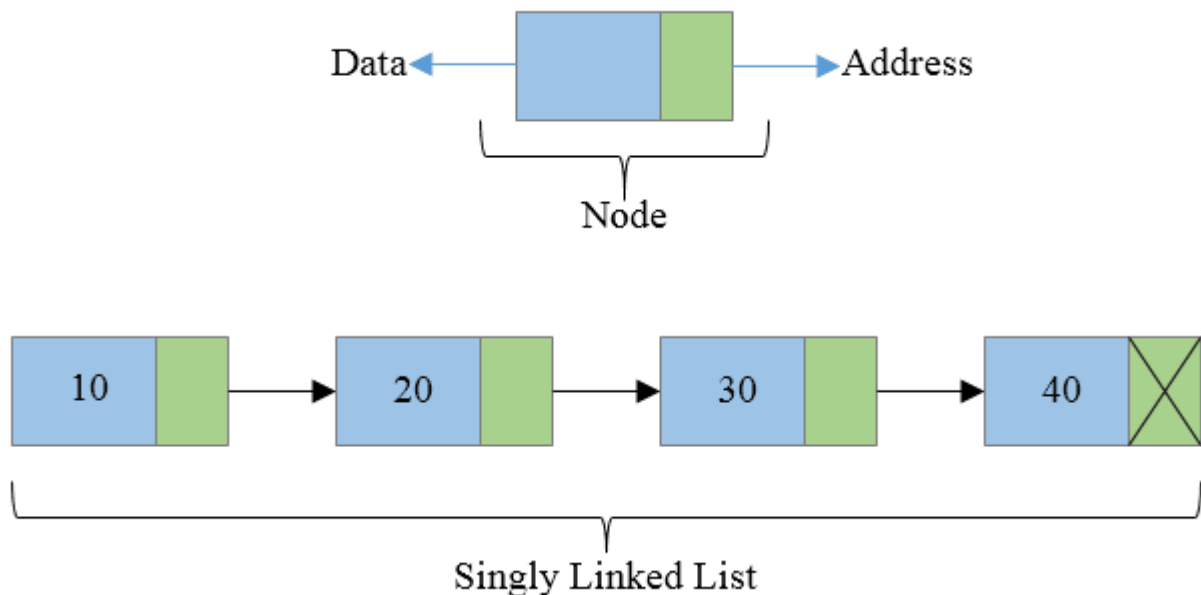


Data Structure : Singly Linked list

Singly linked list is a basic linked list type. Singly linked list is a collection of nodes linked together in a sequential way where each node of singly linked list contains a data field and an address field which contains the reference of the next node. Singly linked list can contain multiple data fields but should contain at least single address field pointing to its connected next node.



To perform any operation on a linked list we must keep track/reference of the first node which may be referred by head pointer variable. In singly linked list address field of last node must contain a NULL value specifying end of the list.

Basic structure of a singly linked list

Each node of a singly linked list follows a common basic structure. In a node we can store more than one data fields but we need at least single address field to store the address of next connected node.

```
struct node {
    int data;           // Data
    struct node * next; // Address
};
```

Advantages of Singly linked list

There are several points about singly linked list that makes it an important data structure.

- Singly linked list is probably the most easiest data structure to implement.
- Insertion and deletion of element can be done easily.
- Insertion and deletion of elements doesn't requires movement of all elements when compared to an array.
- Requires less memory when compared to doubly, circular or doubly circular linked list.
- Can allocate or deallocate memory easily when required during its execution.
- It is one of most efficient data structure to implement when traversing in one direction is required.

Disadvantages of Singly linked list

After seeing the advantages of singly linked list. Singly linked list also has some disadvantages over other data structures.

- It uses more memory when compared to an array.
- Since elements are not stored sequentially hence requires more time to access each elements of list.
- Traversing in reverse is not possible in case of Singly linked list when compared to Doubly linked list.
- Requires $O(n)$ time on appending a new node to end. Which is relatively very high when compared to array or other linked list.

/*Singly linked list implementation using different functions

- 1.INSERT A NUMBER AT THE BEGINNING;
- 2.INSERT A NUMBER AT LAST
- 3.INSERT A NUMBER AT A PARTICULAR LOCATION IN LIST
- 4.PRINT THE ELEMENTS IN THE LIST
- 5.PRINT THE TOTAL NUMBER OF ELEMENTS IN THE LIST
- 6.DELETE A NODE IN THE LINKED LIST:
- 7.REVERSE A LINKED LIST */

`/* PROGRAM IMPLEMENTATION OF SINGLE LINKED LIST */`

```
#include<stdio.h>
```

```
/* STRUCTURE CONTANING A DATA PART AND A LINK PART */
```

```
struct node
```

```
{
```

```
int data;
```

```
struct node *next;
```

```
}*p;
```

```
/* P IS A GLOBAL POINTER CONTAINS THE ADRESS OF THE FIRST NODE IN  
LIST
```

```
*/
```

```
/*THIS FUNCTION DELETES A NODE */
```

```
delnode(int num)
```

```
{
```

```
struct node *temp, *m;
```

```
temp=p;
```

```
while(temp!=NULL)
```

```
{
```

```
if(temp->data==num)
```

```
{
```

```
if(temp==p)
```

```
{
```

```
p=temp->next;
```

```
free(temp);
```

```
return;
```

```

}
else
{
m->next=temp->next;
free(temp);
return;
}
}else
{
m=temp;
temp= temp->next;
}

}
printf("ELEMENT %d NOT FOUND", num);
}/*THIS FUNCTION ADDS A NODE AT THE LAST OF LINKED LIST */

```

```

append( int num )
{
struct node *temp,*r;
/* CREATING A NODE AND ASSIGNING A VALUE TO IT */

temp= (struct node *)malloc(sizeof(struct node));
temp->data=num;
r=(struct node *)p;

if (p == NULL) /* IF LIST IS EMPTY CREATE FIRST NODE */
{
p=temp;
p->next =NULL;
}

```

else

```
{ /* GO TO LAST AND ADD*/
```

```
while( r->next != NULL)
```

```
r=r->next;
```

```
r->next =temp;
```

```
r=temp;
```

```
r->next=NULL;
```

```
}
```

```
}/* ADD A NEW NODE AT BEGINNING */
```

```
addbeg( int num )
```

```
{
```

```
/* CREATING A NODE AND INSERTING VALUE TO IT */
```

```
struct node *temp;
```

```
temp=(struct node *)malloc(sizeof(struct node));
```

```
temp->data=num;
```

```
/* IF LIST IS NULL ADD AT BEGINNING */
```

```
if ( p== NULL)
```

```
{
```

```
p=temp;
```

```
p->next=NULL;
```

```
}
```

else

```
{
```

```
temp->next=p;
```

```
p=temp;
```

```
}
```

```
}
```

```
/* ADD A NEW NODE AFTER A SPECIFIED NO OF NODES */
```

```
addafter(int num, int loc)
```

```
{
```

```
int i;
```

```
struct node *temp,*t,*r;
```

```
r=p; /* here r stores the first location */
```

```
if(loc > count()+1 || loc <= 0)
```

```
{
```

```
printf("insertion is not possible :");
```

```
return;
```

```
}
```

```
if (loc == 1)/* if list is null then add at beginning */
```

```
{
```

```
addbeg(num);
```

```
return;
```

```
}
```

```
else
```

```
{
```

```
for(i=1;i<loc;i++)
```

```
{
```

```
t=r; /* t will be holding previous value */
```

```
r=r->next;
```

```
}
```

```
temp=(struct node *)malloc(sizeof(struct node));
```

```
temp->data=num;
```

```
t->next=temp;
```

```
t=temp;
```

```
t->next=r;
```

```
return;
}
/* THIS FUNCTION DISPLAYS THE CONTENTS OF THE LINKED LIST */
```

```
display(struct node *r)
{
    r=p;
    if(r==NULL)
    {
        printf("NO ELEMENT IN THE LIST :");
        return;
    }
    /* traverse the entire linked list */
    while(r!=NULL)
    {
        printf(" -> %d ",r->data);
        r=r->next;
    }
    printf("\n");
}

//THIS FUNCTION COUNTS THE NUMBER OF ELEMENTS IN THE LIST
count()
{
    struct node *n;
    int c=0;
    n=p;
    while(n!=NULL)
    {
        n=n->next;
        c++;
    }
}
```

```

return(c);
}
//THIS FUNCTION REVERSES A LINKED LIST
reverse(struct node *q)
{
struct node *m, *n, *s;

m=q;
n=NULL;
while(m!=NULL)
{
s=n;
n=m;
m=m->next;
n->next=s;
}
p=n;
}
/* THIS IS THE MAIN PROGRAM */
main()
{
int i;
p=NULL;
while(1) /* this is an indefinite loop */
{
printf("1.INSERT A NUMBER AT BEGINNING\n");
printf("2.INSERT A NUMBER AT LAST\n");
printf("3.INSERT A NUMBER AT A PARTICULAR LOCATION IN LIST\n");
printf("4.PRINT THE ELEMENTS IN THE LIST \n");
printf("5.PRINT THE NUMBER OF ELEMENTS IN THE LIST \n");
printf("6.DELETE A NODE IN THE LINKED LIST\n");
printf("7.REVERSE A LINKED LIST \n");

```



```
printf("8.Exit\n");
printf("ENTER YOUR CHOICE:");
scanf("%d",&i); /* ENTER A VALUE FOR SWITCH */

switch(i)
{
case 1:
{
int num;
printf("ENTER THE NUMBER TO BE INSERTED :");
scanf("%d",&num);
addbeg(num);
break;
}
case 2:
{
int num;
printf("ENTER THE NUMBER TO BE INSERTED :");
scanf("%d",&num);
append(num);
break;
}

case 3:
{
int num, loc,k;
printf("ENTER THE ELEMENT TO BE INSERTED :");
scanf("%d",&num);
printf(" ENTER THE LOCATION NUMBER :");
scanf("%d",&loc);
addafter(num,loc);
```

```
break;
} case 4:
{
struct node *n;
printf("THE ELEMENTS IN THE LIST ARE \n");
display(n);
break;
}

case 5:
{
struct node *n;
display(n);
printf(" TOTAL NO OF ELEMENTS IN THE LIST ARE %d\n",count());
break;
} case 6:
{
int num;
printf("ENTER A NUMBER FROM THE LIST :");
scanf("%d",&num);
delNode(num);
break;
}

case 7:
{
reverse(p);
display(p);
break;
}

case 8:
{
```

```
exit(1);  
  
}  
  
/* end if switch */  
  
/* end of while */  
  
/* end of main */
```